

11-01 — Entity Definition

Section	11 — Entities, Relations, and CRUD with TypeORM
Subtopic	11-01 — Entity Definition
Est. Duration	1.5 – 2 hours
Type	Theory Practical TypeScript
Topics covered	 <code>@Entity</code> decorator, entity options, primary key strategies (<code>@PrimaryColumn</code> , <code>@PrimaryGeneratedColumn</code>), column types, column options (nullable, default, unique), TypeScript enums as column types, class-validator integration basics

Learning Objectives — This Subtopic

- Explain what a TypeORM entity is and how the `@Entity` decorator maps a TypeScript class to a database table.
- Distinguish between `@PrimaryColumn` and `@PrimaryGeneratedColumn` and choose the appropriate strategy for different use cases.
- Apply `@Column` with its full option set (type, nullable, default, unique, length, enum) to define table columns accurately.
- Use TypeScript enums as column types to enforce a constrained set of allowed values at both the application and database level.
- Construct a well-structured project layout separating entity files from DataSource configuration (established in Section 10).
- Verify that TypeORM generates the expected SQL DDL from a TypeScript entity class using `synchronize: true` in a development environment.

1. What is a TypeORM Entity?

In relational databases, data lives in *tables*. In your TypeScript application, data lives in *objects*. The job of an ORM is to bridge that gap — keeping the two in sync without forcing you to write repetitive SQL DDL by hand.

A **TypeORM entity** is a TypeScript class decorated with `@Entity()`. TypeORM reads that decorator at runtime and treats the class as the canonical description of a database table: its name, columns, types, constraints, and relationships. Every property annotated with a column decorator becomes a column; every instance of the class corresponds to a row.

Analogy: Think of an entity class as a *blueprint* for a spreadsheet tab. The class name (or the name you give `@Entity()`) becomes the tab name; each decorated property becomes a column header; each saved instance of the class fills in a row. The ORM uses that blueprint to both create the tab and read/write data to it.

Prerequisites from earlier in the module:

- **Section 10** covered installing TypeORM, configuring a `DataSource`, and connecting to a database. The `AppDataSource` pattern used there is assumed throughout these notes.
- TypeScript compilation is assumed to be configured (`tsconfig.json` with `experimentalDecorators: true` and `emitDecoratorMetadata: true`, installed in Section 10).

Note: TypeORM decorators rely on the TypeScript compiler emitting metadata about property types. Without `"emitDecoratorMetadata": true` in `tsconfig.json`, TypeORM cannot infer column types from TypeScript types alone. This was configured in Section 10 — verify it is present before proceeding.

2. The `@Entity` Decorator

The `@Entity()` decorator (imported from `typeorm`) marks a class as a managed entity. At a minimum it takes no arguments — TypeORM will use the lowercased class name as the table name. You can pass a string to override that name, or a full options object for finer control.

2.1 Minimal Entity

STAGE 1 The smallest possible entity — one class, one decorator, no columns yet.

src/entity/Product.ts

```
import { Entity } from "typeorm";

@Entity()
export class Product {
  // columns will be added in Stage 2
}
```

Output:

No output — this file is a class definition. TypeORM reads it when the DataSource is initialised. When synchronize: true is active, it will create an empty table called `product` in the database.

With no arguments, TypeORM derives the table name by lowercasing the class name: `Product` → `product`. This is a sensible convention for simple cases.

2.2 Providing a Custom Table Name

STAGE 2 Pass a string argument to control the table name — useful when the class name and the database naming convention differ.

src/entity/Product.ts

```
import { Entity } from "typeorm";

@Entity("products") // table will be named "products" (plural)
```

```
export class Product {  
  // ...  
}
```

Output:

TypeORM will create / map to a table named **products** instead of **product**.

2.3 Entity Options Object

For advanced scenarios, pass an options object instead of a plain string:

src/entity/Product.ts

```
import { Entity } from "typeorm";  
  
@Entity({  
  name: "products",           // explicit table name  
  schema: "catalogue",       // database schema (PostgreSQL / MSSQL)  
  orderBy: { name: "ASC" },   // default ordering when no ORDER BY is given  
})  
export class Product {  
  // ...  
}
```

Output:

The entity maps to **catalogue.products** and rows are ordered by **name ASC** by default.
(Schema option has no effect on SQLite; it is meaningful in PostgreSQL and SQL Server.)

Key Insight: The `name` string shorthand and the options object are the two patterns you will encounter most often. Use the string shorthand when you only need to rename the table; use the object when you need schema control or default ordering.

3. Primary Keys

Every entity must have at least one primary key — the unique identifier for each row. TypeORM provides two decorators for this:

- `@PrimaryColumn()` — you supply the value; TypeORM does not generate it.
- `@PrimaryGeneratedColumn()` — TypeORM (via the database) generates the value automatically.

3.1 `@PrimaryGeneratedColumn` — Auto-increment Integer

STAGE 1 The most common choice: an auto-incrementing integer ID.

src/entity/Product.ts

```
import { Entity, PrimaryGeneratedColumn } from "typeorm";

@Entity("products")
export class Product {
  @PrimaryGeneratedColumn()
  id: number;
}
```

Output:

```
Generated SQL (SQLite / MySQL):
CREATE TABLE products (
  id integer PRIMARY KEY AUTOINCREMENT NOT NULL
);
```

When you `save()` a new `Product` instance, you do not set `id` — the database assigns it and TypeORM writes it back onto your object.

3.2 `@PrimaryGeneratedColumn('uuid')` — UUID Strategy

STAGE 2 Pass `'uuid'` to generate a UUID v4 instead of an integer. This is common for distributed systems where you cannot rely on a single sequence.

src/entity/Product.ts

```
import { Entity, PrimaryGeneratedColumn } from "typeorm";

@Entity("products")
export class Product {
  @PrimaryGeneratedColumn("uuid")
  id: string;    // TypeScript type is now string, not number
}
```

Output:

Generated SQL (PostgreSQL):

```
CREATE TABLE products (
  id uuid PRIMARY KEY DEFAULT uuid_generate_v4() NOT NULL
);
```

3.3 @PrimaryColumn — Manual Primary Key

Use `@PrimaryColumn()` when the primary key value is domain-specific and should be set by your application — for example, an ISIN code for financial instruments, an ISBN for books, or a national identifier.

src/entity/Country.ts

```
import { Entity, PrimaryColumn, Column } from "typeorm";

@Entity("countries")
export class Country {
  @PrimaryColumn({ length: 2 }) // ISO 3166-1 alpha-2, e.g. "GB", "US"
  code: string;

  @Column()
  name: string;
}
```

Output:

Generated SQL:

```
CREATE TABLE countries (  
  code varchar(2) PRIMARY KEY NOT NULL,  
  name varchar(255) NOT NULL  
);
```

Important: With `@PrimaryColumn()`, TypeORM will not generate a value for you. If you `save()` an entity instance without setting the primary key, TypeORM will throw a runtime error. Use `@PrimaryGeneratedColumn()` whenever you want the database to own ID generation.

3.4 Composite Primary Keys

A table can have a composite primary key — multiple columns that together form a unique identifier — by decorating more than one property with `@PrimaryColumn()`.

src/entity/OrderItem.ts

```
import { Entity, PrimaryColumn, Column } from "typeorm";  
  
@Entity("order_items")  
export class OrderItem {  
  @PrimaryColumn()  
  orderId: number;  
  
  @PrimaryColumn()  
  productId: number;  
  
  @Column("int")  
  quantity: number;  
}
```

Output:

Generated SQL:

```
CREATE TABLE order_items (  
  orderId integer NOT NULL,  
  productId integer NOT NULL,  
  quantity integer NOT NULL,  
  PRIMARY KEY (orderId, productId)  
);
```

Note: Composite keys are most useful for pure junction tables. For most entities, a single auto-generated surrogate key (`@PrimaryGeneratedColumn()`) is the recommended default — it simplifies queries and avoids update anomalies.

4. The `@Column` Decorator

Every non-primary property that should be persisted to the database is decorated with `@Column()` . It accepts either a column type string, an options object, or both.

4.1 Implicit Type Inference

STAGE 1 When you pass no arguments, TypeORM infers the column type from the TypeScript property type.

src/entity/Product.ts

```
import { Entity, PrimaryGeneratedColumn, Column } from "typeorm";

@Entity("products")
export class Product {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  name: string;           // inferred as varchar(255)

  @Column()
  price: number;          // inferred as double precision / float

  @Column()
  inStock: boolean;       // inferred as boolean / tinyint(1)
}
```

Output:

```
Generated SQL (MySQL):  
CREATE TABLE products (  
  id      int      PRIMARY KEY AUTO_INCREMENT NOT NULL,  
  name    varchar(255) NOT NULL,  
  price   double    NOT NULL,  
  inStock tinyint   NOT NULL  
);
```

TypeORM maps TypeScript primitives to database types as follows: `string` → `varchar(255)`, `number` → `double/float`, `boolean` → `tinyint(1)` (MySQL) or `boolean` (PostgreSQL). The exact SQL type depends on which database driver is active.

4.2 Explicit Column Type

STAGE 2 Pass a type string when the inferred type is too broad. For example, a monetary value should be stored as `decimal`, not `double`, to avoid floating-point rounding errors.

src/entity/Product.ts

```
import { Entity, PrimaryGeneratedColumn, Column } from "typeorm";  
  
@Entity("products")  
export class Product {  
  @PrimaryGeneratedColumn()  
  id: number;  
  
  @Column()  
  name: string;  
  
  @Column("decimal", { precision: 10, scale: 2 })  
  price: number;      // stored as DECIMAL(10,2) – safe for money  
  
  @Column("int")  
  stockCount: number; // stored as INT, not DOUBLE  
  
  @Column("text")  
  description: string; // stored as TEXT (unlimited length)  
}
```

Output:

Generated SQL (MySQL):

```
CREATE TABLE products (  
  id          int          PRIMARY KEY AUTO_INCREMENT NOT NULL,  
  name        varchar(255) NOT NULL,  
  price       decimal(10,2) NOT NULL,  
  stockCount  int          NOT NULL,  
  description text         NOT NULL  
);
```

Key Insight: Always use "decimal" (with precision and scale) for currency or any value where rounding matters. Floating-point types (double, float) cannot represent all decimal fractions exactly — $0.1 + 0.2$ is a classic example of a value that cannot be stored precisely in binary floating point.

4.3 Column Options

STAGE 3 The full options object gives you fine-grained control over every aspect of the column.

src/entity/Product.ts

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn, UpdateDateColumn } from  
"typeorm";  
  
@Entity("products")  
export class Product {  
  @PrimaryGeneratedColumn()  
  id: number;  
  
  @Column({ length: 120, unique: true })  
  name: string;           // varchar(120), UNIQUE constraint  
  
  @Column("decimal", { precision: 10, scale: 2 })  
  price: number;  
  
  @Column({ nullable: true, type: "text" })  
  description: string | null; // column allows NULL  
  
  @Column({ default: true })  
  isActive: boolean;        // DEFAULT TRUE at database level  
  
  @Column({ default: 0, type: "int", unsigned: true })  
  stockCount: number;       // DEFAULT 0, no negative values
```

```
@Column({ name: "sku_code", length: 50, nullable: true })
skuCode: string | null;      // column in DB is named "sku_code"

@CreateDateColumn()
createdAt: Date;             // automatically set on INSERT

@UpdateDateColumn()
updatedAt: Date;             // automatically updated on UPDATE
}
```

Output:

Generated SQL (PostgreSQL):

```
CREATE TABLE products (
  id          SERIAL          PRIMARY KEY,
  name        varchar(120)    NOT NULL UNIQUE,
  price       numeric(10,2)   NOT NULL,
  description  text,
  isActive    boolean        NOT NULL DEFAULT true,
  stockCount  integer        NOT NULL DEFAULT 0,
  sku_code    varchar(50),
  createdAt   TIMESTAMP      NOT NULL DEFAULT now(),
  updatedAt   TIMESTAMP      NOT NULL DEFAULT now()
);
```

The key options used above, summarised:

Option	Type	Effect
length	number	Sets <code>varchar(n)</code> length
unique	boolean	Adds a UNIQUE constraint to the column
nullable	boolean	Allows NULL values; defaults to <code>false</code>
default	any	Database-level DEFAULT value
unsigned	boolean	UNSIGNED integer (MySQL / SQLite)
name	string	

Option	Type	Effect
		Overrides the database column name
<code>precision</code>	number	Total digits for <code>decimal</code>
<code>scale</code>	number	Decimal places for <code>decimal</code>

Key Insight: `@CreateDateColumn()` and `@UpdateDateColumn()` are convenience decorators built on top of `@Column`. They hook into TypeORM's lifecycle to set/update timestamps automatically — you never write to these properties directly.

5. TypeScript Enums as Column Types

A common requirement is to restrict a column to a fixed set of string values — for example, a product's status might only ever be `"draft"`, `"active"`, or `"discontinued"`. TypeScript enums combine well with TypeORM's `enum` column type to enforce this at both the application layer (TypeScript type-checking) and the database layer (a CHECK constraint or native ENUM type, depending on the driver).

5.1 Defining and Using an Enum Column

src/entity/Product.ts

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn, UpdateDateColumn } from
"typeorm";

// Defined outside the class so it can be imported by other modules
export enum ProductStatus {
  Draft      = "draft",
  Active     = "active",
  Discontinued = "discontinued",
}

@Entity("products")
export class Product {
```

```
@PrimaryGeneratedColumn()
id: number;

@Column({ length: 120, unique: true })
name: string;

@Column("decimal", { precision: 10, scale: 2 })
price: number;

@Column({
  type: "enum",
  enum: ProductStatus,
  default: ProductStatus.Draft,
})
status: ProductStatus;

@Column({ nullable: true, type: "text" })
description: string | null;

@Column({ default: 0, type: "int" })
stockCount: number;

@CreateDateColumn()
createdAt: Date;

@UpdateDateColumn()
updatedAt: Date;
}
```

Output:

Generated SQL (MySQL):

```
CREATE TABLE products (
  id          int          PRIMARY KEY AUTO_INCREMENT NOT NULL,
  name        varchar(120) NOT NULL UNIQUE,
  price       decimal(10,2) NOT NULL,
  status      enum('draft','active','discontinued') NOT NULL DEFAULT 'draft',
  description text,
  stockCount  int          NOT NULL DEFAULT 0,
  createdAt   datetime     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updatedAt   datetime     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

The database now enforces that `status` can only ever hold one of the three enum values. Attempting to insert any other string will result in a database-level error — a useful defence-in-depth layer even if your application-level validation fails.

Note: Always use string-valued enums (`= "draft"`) rather than numeric enums (`= 0`) for database columns. String values are human-readable in the database, survive refactoring better, and are far easier to migrate if the enum grows. Numeric enum values have no inherent meaning in the stored data.

Important — SQLite limitation: SQLite does not have a native ENUM type. TypeORM falls back to storing enum columns as `varchar` with a CHECK constraint. The behaviour is functionally identical, but be aware that the generated SQL will differ from MySQL / PostgreSQL when reviewing schema output.

6. Project Layout and Registering Entities with DataSource

An entity file is just a TypeScript class. TypeORM only manages it once it is registered in the `entities` array of your `DataSource` configuration (covered in Section 10). Below is a recommended project structure and the wiring needed to make everything work.

```
src/
├─ data-source.ts      ← DataSource configuration (from Section 10)
├─ entity/
│   ├─ Product.ts      ← Entity defined in this subtopic
│   └─ Country.ts
├─ index.ts            ← Application entry point
└─ tsconfig.json
```

`src/data-source.ts`

```
import "reflect-metadata";
import { DataSource } from "typeorm";
import { Product } from "../entity/Product";
import { Country } from "../entity/Country";

export const AppDataSource = new DataSource({
  type: "sqlite",
  database: "dev.db",
  synchronize: true,          // auto-creates / alters tables from entities – dev only
  logging: true,              // logs generated SQL to the console
  entities: [Product, Country],
});
```

Output:

No runtime output – this is configuration. Output appears when the DataSource is initialised (see index.ts below).

src/index.ts

```
import { AppDataSource } from "../data-source";

AppDataSource.initialize()
  .then(() => {
    console.log("Data Source has been initialised.");
    // At this point, TypeORM has read the entity classes and created the tables.
  })
  .catch((err) => {
    console.error("Error during Data Source initialisation:", err);
  });
```

Compile and run with:

```
npx ts-node src/index.ts
```

Output:

```
query: CREATE TABLE IF NOT EXISTS "products" ("id" integer PRIMARY KEY AUTOINCREMENT NOT NULL, "name"
varchar(120) NOT NULL UNIQUE, "price" decimal(10,2) NOT NULL, "status" varchar NOT NULL DEFAULT 'draft',
"description" text, "stockCount" integer NOT NULL DEFAULT (0), "createdAt" datetime NOT NULL DEFAULT
(datetime('now')), "updatedAt" datetime NOT NULL DEFAULT (datetime('now')))
```

```
query: CREATE TABLE IF NOT EXISTS "countries" ("code" varchar(2) PRIMARY KEY NOT NULL, "name" varchar(255) NOT NULL)
Data Source has been initialised.
```

Important — `synchronize: true` in production: The `synchronize` option automatically modifies your database schema to match your entity definitions every time the application starts. In development this is convenient. In production it is dangerous — it can silently drop columns or tables if you rename a property. Section 12 covers **Migrations**, which are the correct mechanism for production schema changes. Always set `synchronize: false` in production environments.

6.1 Wildcard Entity Registration

For larger projects, enumerating every entity in the `entities` array becomes tedious. TypeORM accepts glob patterns:

src/data-source.ts (alternative)

```
import { DataSource } from "typeorm";

export const AppDataSource = new DataSource({
  type: "sqlite",
  database: "dev.db",
  synchronize: true,
  logging: true,
  entities: [__dirname + "/entity/**/*.ts"], // picks up all entity files automatically
});
```

Output:

Equivalent result – TypeORM discovers and registers all entity files in `src/entity/` without manual imports.

7. Column Naming and the `name` Option

By default, TypeORM uses the property name as the column name. TypeScript convention favours `camelCase` property names, but many database conventions (and DBAs) prefer `snake_case` column names. You have two options to handle this mismatch:

7.1 Per-column Override

src/entity/Product.ts (excerpt)

```
@Column({ name: "sku_code", length: 50, nullable: true })
skuCode: string | null;
```

Output:

Column in DB: `sku_code` | Property in TypeScript: `skuCode`
TypeORM maps between them transparently on read and write.

7.2 Global Naming Strategy

For a consistent project-wide convention, implement a custom `NamingStrategy`. TypeORM ships with a `SnakeNamingStrategy` you can apply once and forget:

src/data-source.ts

```
import { DataSource } from "typeorm";
import { SnakeNamingStrategy } from "typeorm-naming-strategies"; // npm install typeorm-naming-strategies

export const AppDataSource = new DataSource({
  type: "sqlite",
  database: "dev.db",
  synchronize: true,
  logging: true,
  entities: [__dirname + "/entity/**/*.ts"],
  namingStrategy: new SnakeNamingStrategy(),
  // Now: stockCount → stock_count, createdAt → created_at, etc.
});
```

Output:

All `camelCase` property names are automatically converted to `snake_case` column names across the entire project. No per-column name option is needed.

Note: `typeorm-naming-strategies` is a lightweight third-party package — install it with `npm install typeorm-naming-strategies`. It is not part of TypeORM core. If you prefer to avoid the dependency, use per-column `name` overrides where needed.

Summary

Concept	Key Point
<code>@Entity()</code>	Marks a TypeScript class as a managed entity; maps it to a database table. Accepts a name string or options object.
<code>@PrimaryGeneratedColumn()</code>	Auto-incrementing integer primary key (default) or UUID (<code>'uuid'</code> argument). Database generates the value.
<code>@PrimaryColumn()</code>	Application-supplied primary key. Required when the domain provides the identifier (e.g. ISO codes, ISBNs).
Composite PK	Multiple <code>@PrimaryColumn()</code> decorators on one entity produce a composite primary key.
<code>@Column()</code> type inference	Without arguments, TypeORM infers the column type from the TypeScript property type. Explicit types override this.
<code>nullable</code> option	

Concept	Key Point
	Defaults to <code>false</code> (NOT NULL). Set <code>nullable: true</code> to allow NULL values; update the TypeScript type to <code>T null</code> .
<code>default</code> option	Sets a database-level DEFAULT value. Distinct from a TypeScript property initialiser, which only applies in application code.
Enum columns	Pass <code>{ type: "enum", enum: YourEnum }</code> to constrain a column to a fixed set of string values.
<code>@CreateDateColumn</code> / <code>@UpdateDateColumn</code>	Convenience decorators that automatically manage timestamp columns. No manual assignment required.
<code>synchronize: true</code>	Auto-syncs database schema from entities on start-up. Development only — use migrations in production (Section 12).
Naming strategy	Use <code>name</code> option per-column or a global <code>SnakeNamingStrategy</code> to map <code>camelCase</code> TS properties to <code>snake_case</code> DB columns.

Interview Questions

1. What does decorating a class with `@Entity()` do, and what TypeScript compiler options are required for it to function?

Expected answer: It marks the class as a managed entity, instructing TypeORM to map it to a database table and manage its lifecycle. The TypeScript compiler must have `experimentalDecorators: true` and `emitDecoratorMetadata: true` in `tsconfig.json`; without them, decorator metadata (including property types) is not emitted and TypeORM cannot infer column types automatically.

2. What is the difference between `@PrimaryGeneratedColumn()` and `@PrimaryColumn()` ? Give an example of when you would choose each.

Expected answer: `@PrimaryGeneratedColumn()` delegates ID generation to the database (auto-increment integer or UUID). `@PrimaryColumn()` requires the application to supply the value. Choose `@PrimaryGeneratedColumn()` for most entities where the database owns identity; choose `@PrimaryColumn()` when the domain provides a meaningful identifier, such as an ISO country code or a product barcode.

3. Why should monetary values be stored as `decimal` rather than the default `double` ?

Expected answer: Floating-point types (`double` , `float`) use binary fractions internally and cannot represent all decimal values exactly. This causes rounding errors that accumulate in financial calculations. `DECIMAL(p, s)` stores values as exact decimal strings, guaranteeing precision to `s` decimal places.

4. What is the effect of `nullable: true` on both the database column and the TypeScript property type?

Expected answer: At the database level it removes the NOT NULL constraint, allowing the column to store NULL. In TypeScript, the property type should be updated to `T | null` to keep the type system honest — otherwise TypeScript assumes the value is always defined but the database may return `null` , causing runtime errors.

5. How do TypeScript string enums improve data integrity compared to storing plain strings in the database?

Expected answer: The enum provides two layers of constraint: at compile time, TypeScript rejects any value not in the enum; at runtime, the database enforces the constraint via a CHECK constraint or native ENUM type. This means invalid values cannot enter the system even if application-level validation is bypassed.

6. What is the difference between `@CreateDateColumn` and a regular `@Column({ default: () => "CURRENT_TIMESTAMP" })` ?

Expected answer: `@CreateDateColumn()` is a high-level TypeORM abstraction that sets the timestamp only on INSERT and maps the result back as a JavaScript `Date` object. A raw column with a SQL default expression requires you to specify the expression manually and still returns the column value as a string from the driver; you would need to convert it yourself. `@CreateDateColumn` handles all of this transparently.

7. Why is `synchronize: true` dangerous in production, and what is the correct alternative?

Expected answer: `synchronize: true` modifies the database schema to match entity definitions every time the application starts. If a property is renamed or removed, TypeORM may drop the corresponding column — permanently deleting data. The correct alternative is **migrations** (covered in Section 12): versioned, reviewable, reversible SQL scripts that are applied explicitly rather than automatically.

8. You have a `User` entity with a `username` property. You rename it to `loginName` in TypeScript but forget to add `name: "username"` to the `@Column` decorator. What happens when `synchronize: true` is active?

Expected answer: TypeORM sees a new column called `loginName` that does not exist in the database, and an old column called `username` that no longer exists in the entity. With `synchronize`, it will add `loginName` and drop `username` — losing all existing usernames. The correct fix is to add `@Column({ name: "username" })` to the renamed property, or to write a migration that renames the column in SQL.

Practical Assignment — Defining a Product Catalogue Entity

Objective: Create a TypeScript project with TypeORM, define a fully annotated `Product` entity, and verify that TypeORM generates the correct database schema from your class definition.

Setup

If you completed the Section 10 practical, reuse that project. Otherwise, initialise a fresh one:

```
mkdir product-catalogue && cd product-catalogue
```

```
npm init -y
```

```
npm install typeorm reflect-metadata sqlite3
```

```
npm install --save-dev typescript ts-node @types/node
```

```
npx tsc --init
```

In `tsconfig.json`, ensure these compiler options are set:

`tsconfig.json` (relevant options)

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true,
    "strictPropertyInitialization": false,
    "outDir": "./dist"
  }
}
```

Step-by-step Instructions

1. Create the project structure:

```
product-catalogue/
├── src/
│   ├── entity/
│   │   └── Product.ts
│   ├── data-source.ts
│   └── index.ts
├── package.json
└── tsconfig.json
```

2. Create the `ProductStatus` enum and `Product` entity in `src/entity/Product.ts` with the following requirements:

- Auto-generated integer primary key.
- A `name` column: `varchar(120)`, unique, not nullable.
- A `price` column: `decimal(10,2)`, not nullable.
- A `status` column: enum type with values `draft`, `active`, `discontinued`; default `draft`.
- A `description` column: `text`, nullable.
- A `stockCount` column: `int`, default 0.

- `createdAt` and `updatedAt` timestamp columns managed automatically.

3. Configure the `DataSource` in `src/data-source.ts`. Use SQLite with `database: "catalogue.db"`, `synchronize: true`, and `logging: true`. Register the `Product` entity.
4. Write the entry point in `src/index.ts`: initialise the `AppDataSource` and log a success message. Import `reflect-metadata` at the very top.
5. Run the application:

```
npx ts-node src/index.ts
```

6. Inspect the generated database using the SQLite CLI:

```
sqlite3 catalogue.db ".schema products"
```

Expected Output

Output:

```
query: CREATE TABLE "products" ("id" integer PRIMARY KEY AUTOINCREMENT NOT NULL, "name" varchar(120) NOT NULL UNIQUE, "price" decimal(10,2) NOT NULL, "status" varchar NOT NULL DEFAULT 'draft', "description" text, "stockCount" integer NOT NULL DEFAULT (0), "createdAt" datetime NOT NULL DEFAULT (datetime('now')), "updatedAt" datetime NOT NULL DEFAULT (datetime('now')))  
Data Source has been initialised.
```

Running `sqlite3 catalogue.db ".schema products"` should print the full CREATE TABLE statement above.

Bonus Challenge

Add a second entity, `Category`, with an auto-generated UUID primary key, a unique `slug` field (`varchar(80)`), and a `label` field (`varchar(120)`). Register it in the `DataSource`, re-run the application, and verify that a `categories` table is created alongside `products`.

Additionally, implement a global `SnakeNamingStrategy` (install `npm install typeorm-naming-strategies`) and observe how `stockCount` and `createdAt` are mapped to `stock_count` and `created_at` in the generated SQL.



Module 4: Node.js, Express & TypeORM — Section 11, Subtopic 01 — Entity Definition

Next: **11-02 Relations** — One-to-One, One-to-Many, Many-to-Many decorators and join strategies

